

ASCALE

Importateur de matériaux de prestige

RAPPORT DE STAGE

Stage de fin de formation — BUT Informatique · IUT de Villetaneuse

Stagiaire	Salma [NOM] — BUT Informatique 2ème année
Établissement	IUT de Villetaneuse — Université Sorbonne Paris Nord
Maître de stage	Nasser [NOM] — Responsable technique, Ascale
Tutrice pédagogique	Pascale Hellegouarch — IUT de Villetaneuse
Période de stage	Avril - Juin 2026 (10 semaines)
Projet principal	Application web de gestion commerciale — Flask · SQLite · Python

Année universitaire 2025 – 2026

Remerciements

*Je tiens à remercier sincèrement **Nasser**, mon maître de stage au sein d'Ascale, pour m'avoir accordé sa confiance dès le premier jour. Sa disponibilité, ses conseils avisés et sa pédagogie bienveillante m'ont permis de progresser à un rythme soutenu et de comprendre les exigences du monde professionnel du développement logiciel. Il a su me guider tout en me laissant l'autonomie nécessaire pour expérimenter, apprendre de mes erreurs et proposer mes propres solutions.*

*Je tiens à remercier sincèrement **Madame Pascale Hellegouarch**, ma tutrice pédagogique à l'IUT de Villetaneuse, pour son suivi attentif tout au long de cette période de stage. Ses retours constructifs et ses encouragements ont été une ressource inestimable pour structurer mon travail et préparer ce rapport avec rigueur.*

*Je tiens à remercier sincèrement **l'ensemble des enseignants et enseignantes de l'IUT de Villetaneuse**, dont la qualité de l'enseignement — en algorithmique, bases de données, développement web et gestion de projet — m'a fourni les fondations solides sur lesquelles j'ai pu m'appuyer tout au long de ce stage.*

*Je tiens à remercier sincèrement **toute l'équipe d'Ascale** pour leur accueil chaleureux et leur esprit de collaboration. Cette immersion au sein d'une équipe agile dynamique m'a confortée dans mon choix de carrière.*

Sommaire

Introduction	4
1. Présentation de l'entreprise	5
1.1 Secteur d'activité et positionnement	5
1.2 Produits et clientèle	5
1.3 Organisation interne	6
1.4 Contexte de la transformation numérique	6
2. Déroulement du stage	7
2.1 Intégration dans l'équipe	7
2.2 Méthodologie Agile / Scrum	7
2.3 Daily meeting matinal	8
2.4 Azure DevOps	8
2.5 Planning des sprints	9
3. Le projet confié et tâches réalisées	10
3.1 Contexte et objectifs	10
3.2 Architecture complète de l'application	10
3.3 Front-end	12
3.4 Back-end	13
3.5 Base de données et schéma	14
3.6 API REST	15
3.7 Déploiement prévu	16
3.8 Système de paiement simulé et facturation	17
3.9 Chatbot LLM — Mode hybride	19
3.10 Tableau récapitulatif des tâches	20
4. Outils et méthodes de travail	21
4.1 Stack technique	18
4.2 Environnement de développement	19
4.3 Gestion de version et collaboration	19
4.4 Pratiques de qualité logicielle	19
5. Défis rencontrés	20
6. Bilan des compétences acquises	22
6.1 Compétences techniques	22
6.2 Compétences méthodologiques	23
6.3 Compétences transversales	23
Conclusion	24
Glossaire	25
Annexes	27

Introduction

Dans le cadre de ma formation en deuxième année de BUT Informatique à l'IUT de Villetaneuse, j'ai effectué un stage de dix semaines au sein de la société Ascale, spécialisée dans l'importation et la commercialisation de matériaux de prestige — marbres, granits, onyx et travertins. Ce stage constitue une étape déterminante pour consolider mes acquis théoriques par une expérience professionnelle concrète.

L'entreprise m'a confié une mission ambitieuse : concevoir et développer de bout en bout une application web de gestion commerciale. Là où Ascale fonctionnait avec des outils artisanaux (fichiers Excel, messages WhatsApp manuels), l'objectif était de proposer une solution numérique robuste, intuitive et évolutive couvrant le catalogue produits, les commandes, les réservations showroom, un chatbot de conseil et un tableau de bord analytique.

Tout au long de ces dix semaines, j'ai travaillé au sein d'une équipe qui pratique la méthode Scrum et utilise Azure DevOps pour la gestion de projet. Ce rapport rend compte de l'ensemble de mon expérience : présentation de l'entreprise, déroulement du stage, architecture technique, fonctionnalités développées, défis surmontés et bilan de compétences.

1. Présentation de l'entreprise

1.1 Secteur d'activité et positionnement

Ascale est une entreprise spécialisée dans l'importation, la distribution et la commercialisation de matériaux de construction haut de gamme : marbre, granit, onyx et travertin. Elle s'adresse à une clientèle d'architectes d'intérieur, de promoteurs immobiliers, d'artisans du bâtiment et de particuliers exigeants.

Positionnée sur le segment premium, Ascale se distingue par la qualité de sa sélection de pierres naturelles issues des plus grandes carrières mondiales (Italie, Espagne, Iran, Inde, Turquie, Grèce, Portugal) et par l'accompagnement personnalisé proposé à chaque client.

1.2 Produits et clientèle

Famille	Références (exemples)	Usages typiques	Prix indicatif
Marbre	Blanc Carrare, Noir Marquina, Crema Marfil, Thassos, Bardiglio (10 références au catalogue)	Sol, plan de travail, habillage mural	250 – 1 380 MAD/m ²
Granit	Noir Absolu, Gris Baltic Brown, Rouge India, Bleu Bahia	Plan de travail cuisine, sol à fort trafic	480 – 780 MAD/m ²
Onyx	Blanc Translucide, Vert Malachite, Miel Doré	Revêtement mural rétroéclairé, décoratif	1 800 – 2 800 MAD/m ²
Travertin	Beige, Noce, Silver	Sol intérieur/extérieur, terrasse, salle de bain	280 – 350 MAD/m ²

1.3 Organisation interne

Ascale est organisée autour de trois pôles : commercial (devis, relation client), logistique (stock, livraisons) et technique (développement numérique, showroom digital). Mon stage s'est déroulé au sein du pôle technique, sous la responsabilité directe de Nasser, responsable des projets informatiques.

1.4 Contexte de la transformation numérique

À mon arrivée, Ascale gérait ses commandes manuellement via des feuilles de calcul et des échanges WhatsApp. La digitalisation des processus commerciaux — catalogue en ligne, prise de commande, chatbot de conseil, tableau de bord analytique — était devenue une priorité stratégique pour accompagner la croissance de l'entreprise.

2. Déroulement du stage

2.1 Intégration dans l'équipe

La première semaine a été consacrée à la découverte de l'environnement de travail. Nasser m'a présenté l'entreprise, ses enjeux et ses clients, avant de me donner accès aux outils : dépôt Git Azure DevOps, board Scrum et documentation technique.

J'ai ensuite effectué une analyse de l'existant : étude du prototype Flask initial (stockage en mémoire), identification des lacunes fonctionnelles et techniques, et proposition d'un plan d'action présenté lors d'une réunion de cadrage.

2.2 Méthodologie Agile / Scrum

L'équipe suit un cadre Scrum avec des sprints de deux semaines et quatre cérémonies :

- **Sprint Planning** : 1 à 2 h en début de sprint pour sélectionner les user stories et les estimer en story points (échelle Fibonacci).
- **Daily Scrum (9h)** : 15 min quotidiennes — Qu'ai-je fait hier ? Que vais-je faire ? Y a-t-il des blocages ?
- **Sprint Review** : démonstration des fonctionnalités livrées au Product Owner (Nasser).
- **Rétrospective** : bilan collectif — ce qui a bien fonctionné, actions d'amélioration.

Ce que Scrum m'a apporté : La régularité des cérémonies m'a forcée à planifier ma journée de manière explicite et à prendre l'habitude de communiquer proactivement sur mes avancées.

2.3 Daily meeting matinal

Le daily meeting se tenait chaque matin à 9h00. Le Product Owner (Nasser) y participait, permettant d'obtenir rapidement des arbitrages fonctionnels. J'ai progressivement développé une meilleure autonomie dans la gestion de mes tâches et une plus grande confiance dans l'estimation des délais.

2.4 Azure DevOps

- **Boards** : tableau Kanban (Backlog → À faire → En cours → En révision → Terminé). Chaque user story est tracée et liée aux commits correspondants.
- **Repos** : dépôt Git avec branches par fonctionnalité. Les Pull Requests étaient revues par Nasser avant tout merge sur main.
- **Pipelines** : vérification automatique de syntaxe et tests à chaque push.

2.5 Planning des sprints

Sprint	Titre	Tâches principales
Sprint 0 (S1-S2)	Cadrage et analyse	Prise en main, MCD, backlog produit
Sprint 1 (S3-S4)	Back-end et base de données	Migration SQLAlchemy/SQLite, Store, seeding
Sprint 2 (S5-S6)	Commandes et emails	Formulaire commande, confirmation email, réservation
Sprint 3 (S7-S8)	Chatbot et dashboard	10 intentions chatbot, Chart.js, alertes stock
Sprint 4 (S9-S10)	Front-end et finitions	Calculateur surface, filtres catalogue, export CSV, tests

3. Le projet confié et tâches réalisées

3.1 Contexte et objectifs

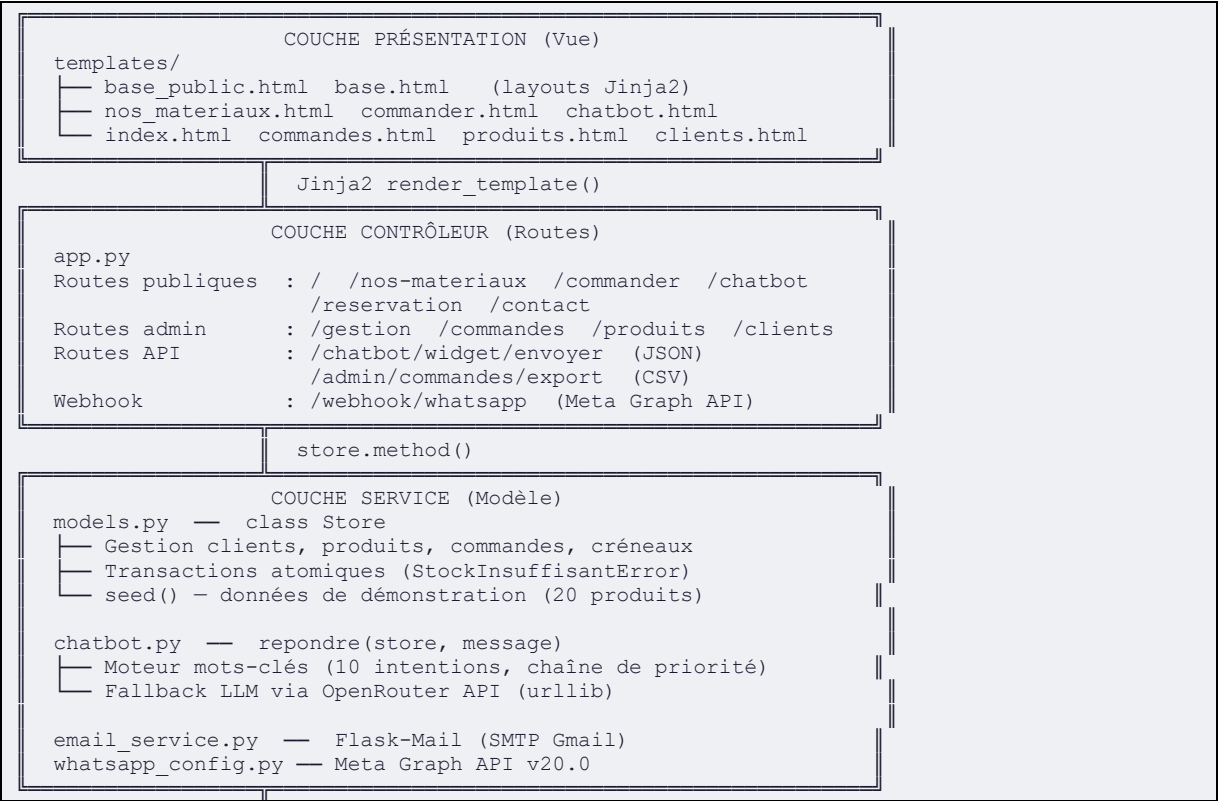
Le cahier des charges demandait une application web complète de gestion commerciale pour Ascale, accessible en deux modes : espace public pour les clients et espace d'administration pour les gestionnaires.

- Permettre aux clients de consulter le catalogue, passer commande et réserver au showroom.
- Offrir aux gestionnaires un tableau de bord avec KPI, alertes stock et graphiques.
- Intégrer un chatbot de conseil (mots-clés + LLM OpenRouter en fallback).
- Automatiser les confirmations par email (commandes, réservations, contact).
- Fournir des exports CSV pour le suivi commercial.
- Tests unitaires complets du back-end (pytest).

3.2 Architecture complète de l'application

L'application suit le patron MVC (Modèle–Vue–Contrôleur) en Python/Flask, structuré en couches clairement séparées.

Schéma 1 — Architecture en couches (MVC)



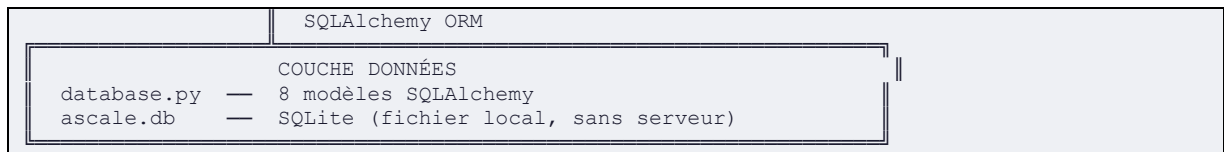


Schéma 2 — Flux des requêtes

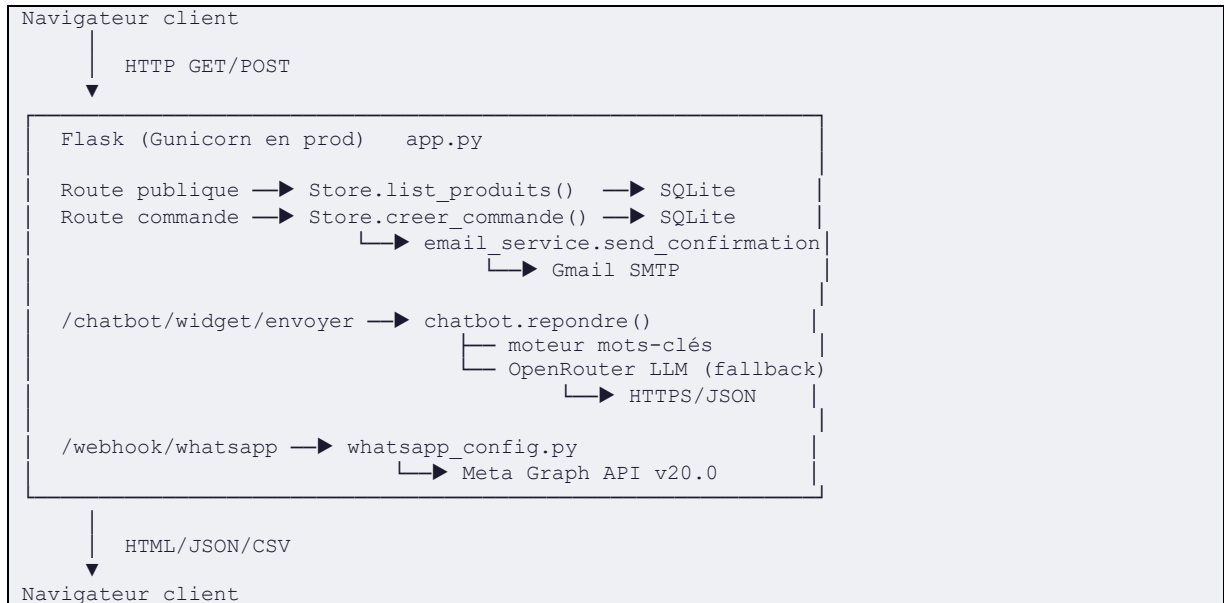
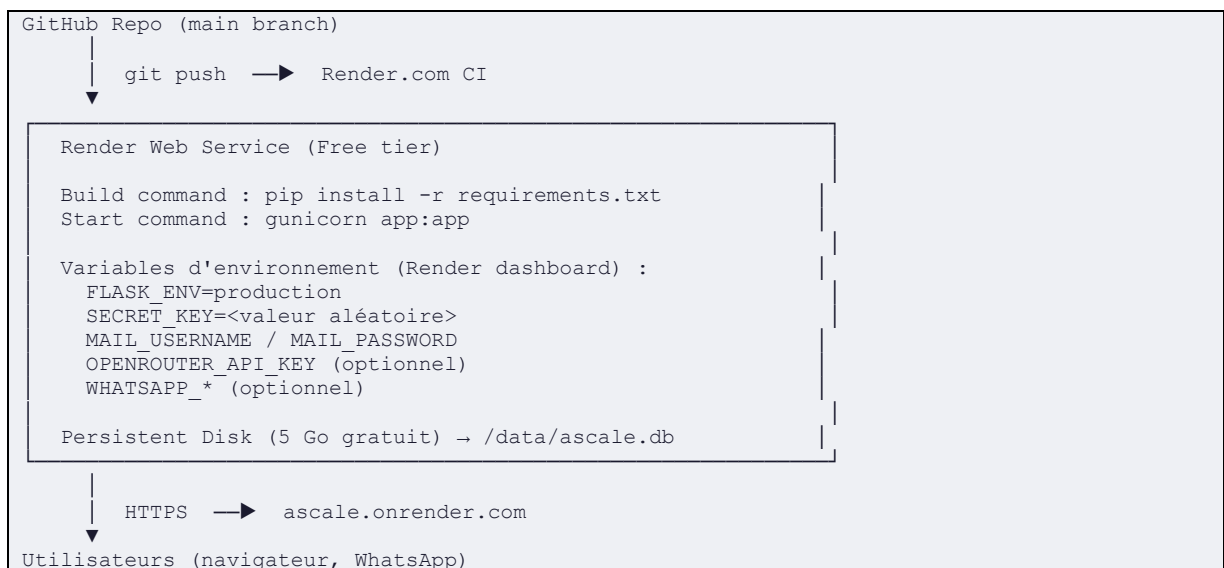


Schéma 3 — Déploiement cible (Render.com)



3.3 Front-end

L'ensemble des interfaces a été développé en Jinja2 + CSS custom + JavaScript natif (ES6), sans framework CSS externe afin de garder la maîtrise totale du design et des performances.

Templates et layouts

- base_public.html — Layout public : navigation, chatbot flottant, footer.
- base.html — Layout admin : sidebar, KPI cards, alerte stock, graphiques Chart.js.
- nos_materiaux.html — Catalogue avec filtres JavaScript (catégorie, disponibilité, prix).
- commander.html — Formulaire commande + calculateur de surface (longueur × largeur × taux de chutes).
- chatbot.html — Interface de simulation chatbot (historique de conversation).
- reservation.html — Sélection de créneaux showroom disponibles.
- contact.html — Formulaire de contact avec auto-réponse chatbot.

CSS et design system

Un système de design custom (palette ivoire/anthracite/or) a été développé avec CSS Grid, Flexbox, variables CSS (--gold, --dark, --light) et media queries pour le responsive mobile-first. Les composants (cards, badges, boutons) sont cohérents entre l'espace public et l'espace admin.

JavaScript natif

- Filtres catalogue — tri par catégorie, disponibilité et prix sans rechargement de page.
- Calculateur de surface — injection automatique du résultat dans les champs de commande.
- Chatbot widget flottant — communication AJAX avec /chatbot/widget/envoyer (JSON).
- Dashboard — graphiques Chart.js (barres top 5 produits, anneau commandes par canal).
- Dropdown statut commandes — soumission automatique du formulaire au changement de valeur.

3.4 Back-end

Flask (app.py)

app.py est le point d'entrée de l'application. Il regroupe toutes les routes HTTP organisées en deux espaces : public (accès libre) et admin (accès restreint). Le pattern create_app(test_config) permet l'injection de configuration en test pour isoler la base de données.

- Validation côté serveur sur chaque POST (format email, longueur, cohérence stocks).
- Flash messages pour retours utilisateur (succès, erreurs).
- Sessions Flask signées (SECRET_KEY) pour l'authentification admin.
- Export CSV avec BOM UTF-8 pour compatibilité Excel.

Chatbot (chatbot.py)

Le moteur chatbot repondre(store, message) analyse le message normalisé (sans accents, minuscules) et le fait passer par une chaîne de 10 intentions ordonnées par priorité. En cas d'échec, un appel LLM via OpenRouter API est tenté.

- Normalisation unicode (suppression accents, lowercasing) pour robustesse.
- Score de correspondance par mots entiers (évite les sous-chaînes).
- LLM fallback : OpenRouter → mistralai/mistral-7b-instruct:free, limité au domaine matériaux.
- Catalogue temps réel injecté dans le system prompt pour des réponses contextualisées.

Emails (email_service.py)

Flask-Mail gère l'envoi SMTP (Gmail TLS) pour trois événements : confirmation de commande, confirmation de réservation showroom, et accusé de réception du formulaire de contact. La configuration MAIL_SUPPRESS_SEND=True désactive l'envoi réel en environnement de test.

Tests unitaires (pytest)

105 tests couvrent la couche modèle (Store), le moteur chatbot et les routes Flask. Chaque test dispose d'une base SQLite in-memory isolée via StaticPool, garantissant une exécution déterministe sans effets de bord.

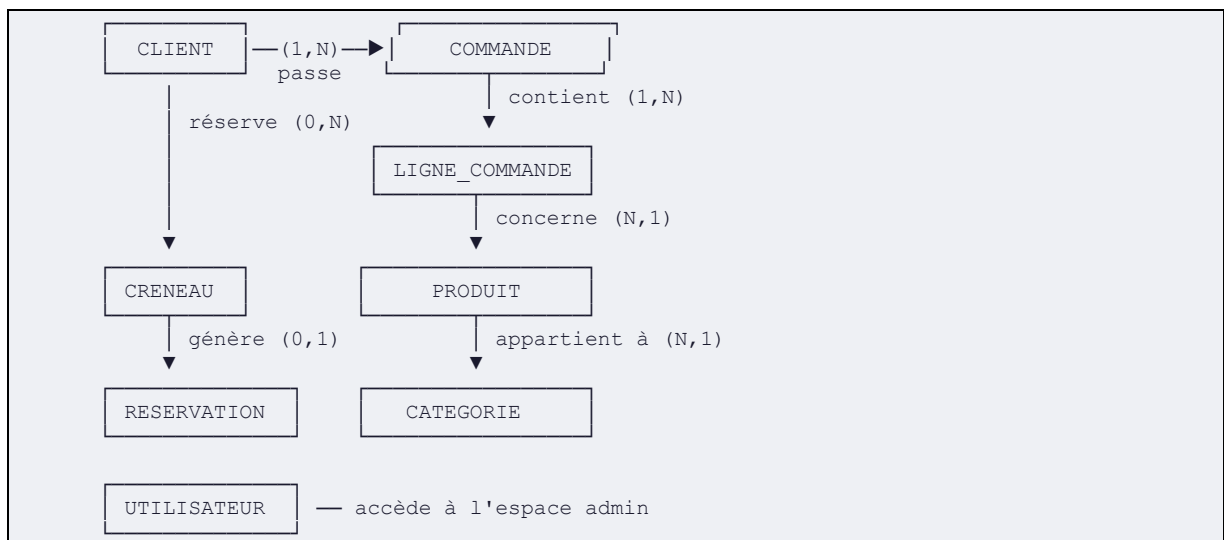
Module de test	Nb tests	Ce qui est testé
test_models.py	42	Store : clients, produits, commandes (atomicité), créneaux
test_chatbot.py	27	10 intentions, faux positifs, fallback LLM (mocké)
test_routes.py	36	Routes GET/POST, codes HTTP, contenu JSON/CSV

3.5 Base de données et schéma

Choix technologique

Pourquoi SQLite ? Simplicité de déploiement (fichier unique ascale.db), compatibilité totale avec SQLAlchemy, adéquation avec le volume de données. Une migration PostgreSQL est facilitée à l'avenir (seul `SQLALCHEMY_DATABASE_URI` change).

Modèle Conceptuel de Données (MCD)



Dictionnaire des entités

Entité	Attributs principaux	Relations
CLIENT	nom, email, téléphone, adresse	Passe → COMMANDE ; Réserve → CRENEAU
PRODUIT	nom, prix/m ² , stock, description, seuil	Appartient à → CATEGORIE ; Ligne → COMMANDE
CATEGORIE	nom	Regroupe → PRODUIT
COMMANDE	date, statut, source (admin/public), total	Contient → LIGNE_COMMANDE
LIGNE_COMMANDE	quantité, prix_unitaire, sous_total	Association COMMANDE ↔ PRODUIT
CRENEAU	date, heure_début, heure_fin, disponible	(0,1) → RESERVATION
RESERVATION	client_nom, email, téléphone, email_envoyé	Réserve → CRENEAU
UTILISATEUR	nom, email, hash_mdp, rôle	Accède à l'espace admin

Points techniques notables

- `db.session.flush()` : écriture partielle sans commit pour obtenir l'ID avant création des lignes.
- Transactions atomiques : vérification de TOUS les stocks avant toute décrémentation.
- StaticPool (tests) : partage de la même connexion in-memory entre tous les accès de test.
- LegacyAPIWarning SQLAlchemy 2.x : `Query.get()` sera migré en `Session.get()` lors de la mise à jour.

3.6 API REST

L'application expose plusieurs endpoints utilisés à la fois par les pages HTML (navigation classique) et par les appels JavaScript asynchrones (AJAX/JSON).

Méthode	URL	Type	Description
GET	/	HTML	Page d'accueil publique
GET	/nos-materiaux	HTML	Catalogue produits (filtres JS)
GET	/commander	HTML	Formulaire commande + calculateur
POST	/commander	HTML	Création commande publique → redirect
GET	/commande/confirmee/<id>	HTML	Page de confirmation commande
GET	/reservation	HTML	Sélection créneaux disponibles
POST	/reservation	HTML	Création réservation → redirect
GET	/contact	HTML	Formulaire de contact
POST	/contact	HTML	Envoi message + auto-réponse chatbot
POST	/chatbot/widget/envoyer	JSON	API chatbot (widget flottant)
GET	/gestion	HTML	Dashboard admin (KPI, graphiques)
GET	/commandes	HTML	Liste des commandes admin
POST	/commandes/<id>/statut	HTML	Mise à jour statut commande
GET	/admin/commandes/export	CSV	Export commandes (UTF-8 BOM)
GET	/admin/clients/export	CSV	Export clients (UTF-8 BOM)
GET	/produits	HTML	Gestion catalogue admin
POST	/produits/ajouter	HTML	Ajout produit
GET	/clients	HTML	Gestion clients admin
GET	/webhook/whatsapp	TEXT	Vérification webhook Meta (challenge)
POST	/webhook/whatsapp	JSON	Réception messages WhatsApp

Convention de réponse chatbot (/chatbot/widget/envoyer) : Entrée : `{"message": "bonjour"}` —

Sortie : {"reponse": "...", "status": "ok"} ou HTTP 400 si message vide.

3.7 Déploiement prévu

La solution de déploiement retenue est Render.com (tier gratuit), choisi pour sa simplicité (déploiement par git push), son support natif de Python/Flask, et l'absence de configuration serveur. C'est la solution la plus rapide pour mettre l'application en production sans coût initial.

Étapes de déploiement

- 1. Ajouter gunicorn aux dépendances : pip install gunicorn → requirements.txt.
- 2. Créer un fichier render.yaml (ou configurer via l'interface Render).
- 3. Connecter le dépôt GitHub à Render (autorisation OAuth).
- 4. Configurer les variables d'environnement dans le dashboard Render.
- 5. Créer un Persistent Disk (5 Go gratuit) monté sur /data/ pour stocker ascale.db.
- 6. Modifier app.py pour pointer la DB vers /data/ascale.db en production.
- 7. Premier déploiement automatique sur git push vers main.

Fichier render.yaml (exemple)

```
services:
- type: web
  name: ascale
  env: python
  buildCommand: pip install -r requirements.txt
  startCommand: gunicorn app:app
  envVars:
    - key: FLASK_ENV
      value: production
    - key: SECRET_KEY
      generateValue: true
    - key: MAIL_USERNAME
      sync: false
    - key: MAIL_PASSWORD
      sync: false
  disk:
    name: ascale-data
    mountPath: /data
    sizeGB: 5
```

Alternatives envisagées

Option	Avantage	Inconvénient
Render.com (retenu)	Simple, git push, gratuit	Mise en veille après 15 min d'inactivité (tier free)
PythonAnywhere	Facile Flask, console SSH	Tier gratuit limité en CPU, sans HTTPS custom
VPS OVH/DigitalOcean	Contrôle total, SSL gratuit	Configuration nginx/gunicorn manuelle, payant
Heroku	Historique Flask, CI/CD natif	Plus de tier gratuit depuis 2022

3.8 Système de paiement simulé et facturation

À l'issue du sprint 5, un tunnel de commande complet a été intégré à l'application. L'objectif était de proposer un parcours d'achat cohérent et professionnel — sans recourir à un prestataire de paiement réel — incluant validation d'adresse, facture PDF automatique et envoi par email.

Tunnel de commande en 3 étapes

Étape	Page Flask	Actions
1 — Sélection	GET/POST /commander	Choix des produits, quantités, coordonnées client. Validation stock côté serveur. Stockage du panier en session Flask.
2 — Paiement	GET/POST /paiement	Adresse de livraison (autocomplétion + validation Nominatim), mode de paiement simulé (CB / virement / espèces). Création commande en base.
3 — Confirmation	GET /commande/recu/<id>	Page reçu avec récapitulatif, adresse, bouton téléchargement facture PDF.

Validation d'adresse via Nominatim (OpenStreetMap)

Pour garantir que l'adresse de livraison est réelle et géolocalisable, l'application effectue deux niveaux de validation :

- Côté client (JavaScript) : autocomplétion en temps réel (debounce 400 ms) via l'API Nominatim. L'utilisateur sélectionne une suggestion ou confirme manuellement.
- Côté serveur (Python) : appel synchrone à Nominatim lors du POST /paiement. Si aucun résultat n'est trouvé, un message d'erreur est affiché. En cas d'indisponibilité de l'API, la validation est non bloquante (fail-open).

Formulaire de paiement simulé

Le formulaire de paiement propose trois modes sans traitement financier réel :

Mode	UI proposée	Comportement serveur
Carte bancaire	Champs numéro (formatage 4×4), titulaire, expiration, CVV — purement visuels. Aucune donnée n'est transmise.	Traite comme un paiement validé. Génère la commande.
Virement bancaire	Affichage du RIB Ascale (simulé).	Génère la commande avec statut « Enregistrée ».
Espèces showroom	Adresse et horaires du showroom.	Génère la commande — règlement différé.

Génération de facture PDF (fpdf2)

Une facture pro forma est générée à la volée après chaque commande validée, à l'aide de la bibliothèque fpdf2 (module `facture_service.py`). Elle comprend en-tête Ascale, informations client, tableau produits avec sous-totaux, total HT, note légale et pied de page. Les bytes PDF sont retournés en mémoire (pas de stockage sur disque) et attachés à l'email de confirmation.

```
# facture_service.py
def generer_facture_pdf(commande, client=None) -> bytes:
    pdf = FPDF()
    pdf.add_page()
    # En-tête Ascale (logo textuel, couleurs charte)
    pdf.set_font("Helvetica", "B", 26)
    pdf.cell(0, 13, "ASCALE", align="C")
    # ... tableau produits, total, note légale ...
    return bytes(pdf.output())
```

Email avec facture en pièce jointe

La fonction `envoyer_facture_commande()` dans `email_service.py` attache les bytes PDF à deux emails : un destiné au client (récapitulatif + facture) et une notification interne à l'équipe Ascale. Si les credentials SMTP ne sont pas configurés dans `.env`, la fonction retourne `False` sans erreur (comportement « fail-silent »).

La page de reçu (`/commande/recu/<id>`) permet de télécharger la facture à tout moment via la route `/commande/facture/<id>.pdf`, qui régénère le PDF à la demande depuis la base de données.

3.9 Chatbot LLM — Mode hybride

Le chatbot d'Ascale a été enrichi d'un toggle « Mode Spécialisé / Mode IA Libre » accessible directement depuis l'interface de chat. Ce mécanisme permet de basculer entre deux comportements distincts sans rechargement de page.

Mode Spécialisé (défaut)

Le mode spécialisé applique la chaîne complète des 10 intentions (salutations, devis, statut commande, budget, recommandations, FAQ, catalogue, catégorie, info produit, politesse) avant de passer au fallback LLM si aucune n'est déclenchée. Ce mode garantit des réponses précises et actualisées sur le catalogue Ascale.

Mode IA Libre

En mode IA Libre, tous les handlers d'intentions sont bypassés. Le message est envoyé directement à OpenRouter (Mistral 7B Instruct) avec le system prompt Ascale (catalogue temps réel injecté). Ce mode répond à n'importe quelle question liée aux matériaux, à la décoration ou à l'architecture. Il nécessite que la clé `OPENROUTER_API_KEY` soit configurée dans `.env`.

Fix normalisation apostrophes

Un bug subtil avait été identifié : la fonction `_normaliser()` ne convertissait pas les apostrophes (U+0027 et U+2019) en espaces. Ainsi, « c'est quoi le marbre » était normalisé en « c'est quoi le marbre » (apostrophe conservée), empêchant le handler de définition de détecter « c est quoi ». La correction consiste à remplacer les apostrophes par des espaces avant toute analyse, ce qui permet aux requêtes naturelles de type « c'est quoi le granit ? » de retourner la définition complète du matériau.

3.10 Tableau récapitulatif des tâches

Tâche	Domaine	Statut	Sprint
Modélisation MCD (Merise)	Architecture	✓	S0
Migration SQLite (SQLAlchemy)	Back-end	✓	S1
Couche service Store + tests	Back-end	✓	S1
Seeding 20 produits (4 catégories)	Back-end	✓	S1
Chatbot enrichi (10 intentions)	Back-end	✓	S3
Fallback LLM OpenRouter	Back-end	✓	S4
Tests unitaires (105 tests)	Back-end	✓	S4
API /chatbot/widget/envoyer (JSON)	API REST	✓	S3
Export CSV commandes & clients	API REST	✓	S4
Webhook WhatsApp Meta (prêt)	API REST	⌚ creds	—
Formulaire commande publique	Front-end	✓	S2
Calculateur de surface (JS)	Front-end	✓	S4
Filtres catalogue (JS natif)	Front-end	✓	S4
Dashboard KPI + Chart.js	Front-end	✓	S3
Réservation showroom + email	Full-stack	✓	S2
Email automatisé (Flask-Mail)	Full-stack	✓	S2
Authentification admin (Werkzeug)	Full-stack	✓	S1
Statut commandes inline (dropdown)	Full-stack	✓	S4
Tunnel paiement 3 étapes (simulé)	Full-stack	✓	S5
Validation adresse Nominatim (JS+PY)	Full-stack	✓	S5
Facture pro forma PDF (fpdf2)	Full-stack	✓	S5
Email facture PDF en pièce jointe	Full-stack	✓	S5
Chatbot toggle Spécialisé / IA Libre	Back-end	✓	S5
Fix normalisation apostrophes chatbot	Back-end	✓	S5
Déploiement Render.com (prévu)	DevOps	⌚	—

4. Outils et méthodes de travail

4.1 Stack technique

Composant	Technologie	Rôle
Langage	Python 3.11	Langage principal back-end
Framework web	Flask 3.0.3	Serveur HTTP, routage, sessions
ORM	Flask-SQLAlchemy 3.1.1	Mapping objet-relationnel
Base de données	SQLite 3	Persistance (fichier local)
Templates	Jinja2	Génération HTML côté serveur
Email	Flask-Mail 0.10.0	Emails SMTP transactionnels
Config	python-dotenv 1.2.2	Variables d'environnement (.env)
Sécurité	Werkzeug	Hash mots de passe PBKDF2
Graphiques	Chart.js 4.4.0 (CDN)	Dashboard analytics interactif
CSS	Vanilla CSS custom	Design ivoire/anthracite/or
JavaScript	Vanilla JS ES6	Filtres, calculateur, chatbot AJAX, Nominatim
LLM fallback	OpenRouter API	Chatbot étendu (mistral-7b, gratuit)
API géo	Nominatim / OpenStreetMap	Validation adresse de livraison
Factures PDF	fpdf2 2.8.7	Génération factures pro forma en mémoire
API externe	Meta Graph API v20.0	WhatsApp Business (webhook prêt)
Tests	pytest 8.3 + pytest-flask	Tests unitaires back-end (105 tests)
Serveur prod	Gunicorn	WSGI server pour déploiement

4.2 Environnement de développement

- VS Code avec extensions Python, Pylance, Jinja, GitLens et SQLite Viewer.
- Environnement virtuel Python (venv) pour l'isolation des dépendances.
- Flask en mode debug pour le rechargement automatique à chaque modification.
- Chrome DevTools pour le débogage CSS/JS et l'inspection des requêtes réseau.

4.3 Gestion de version et collaboration

Gestion de version via Git, dépôt hébergé sur Azure DevOps Repos. Stratégie de branches fonctionnelles (feature/<nom>), fusionnées sur main après Pull Request revue par Nasser. Conventional commits (feat:, fix:, refactor:) pour une historique lisible.

4.4 Pratiques de qualité logicielle

- Validation serveur : format email, quantités > 0, stocks suffisants — avant tout accès BDD.
- Atomicité des transactions : vérification de tous les stocks avant toute décrémentation.
- Exceptions typées métier : StockInsuffisantError, CreneauIndisponibleError.
- Séparation des responsabilités : logique métier dans Store (models.py), jamais dans les routes.
- Sécurité : mots de passe hashés (Werkzeug PBKDF2), sessions signées, .env non versionné.

5. Défis rencontrés

5.1 Migration vers une base de données persistante

Le premier défi a été la migration du stockage en mémoire vers SQLite sans casser les routes existantes. La contrainte était de conserver l'API publique identique (pattern Façade). Cela m'a confrontée aux concepts du cycle de vie des sessions SQLAlchemy, à la différence entre `flush()` et `commit()`, et à la gestion fine des transactions.

Leçon clé : *`db.session.flush()` écrit en transaction sans valider, permettant d'obtenir l'ID auto-incrémenté d'une commande avant le commit final — nécessaire pour créer les lignes de commande en une seule transaction atomique.*

5.2 Chatbot : moteur mots-clés et fallback LLM

Sans accès LLM garanti (coût), j'ai conçu un moteur basé sur la correspondance de mots-clés et une chaîne de 10 intentions. La principale difficulté était la gestion des faux positifs (ex. : « marbre » dans « entretien du marbre » ne doit pas déclencher la liste catalogue). J'ai ensuite intégré OpenRouter comme fallback LLM avec un system prompt contraint au domaine matériaux, garantissant que le LLM ne sorte pas du périmètre métier.

5.3 Intégrité transactionnelle du stock

La gestion du stock posait un problème classique : deux commandes simultanées sur le même produit risquaient de décrémente le stock en dessous de zéro. Solution : valider l'intégralité des lignes avant d'appliquer un quelconque changement. 105 tests unitaires couvrent tous les cas limites (commande valide, stock insuffisant, atomicité).

5.4 Intégration WhatsApp Business API

Les identifiants Meta (PHONE_NUMBER_ID, ACCESS_TOKEN) n'ont pas pu être obtenus dans les délais. J'ai néanmoins implémenté le webhook de vérification (challenge/response), le parsing des payloads JSON et la fonction d'envoi via `urllib.request`, prête à être activée dès réception des credentials. En attendant, la confirmation par email (Flask-Mail) assure l'équivalent fonctionnel.

5.5 Tests unitaires avec base de données isolée

Chaque test devait fonctionner sur une base SQLite vierge sans affecter les autres. La solution : `StaticPool` de SQLAlchemy partage une connexion in-memory unique pour toute la durée d'un test, avec `scope='function'` pour un reset complet entre les tests.

6. Bilan des compétences acquises

6.1 Compétences techniques

Compétence	Niveau atteint	Éléments démontrés
Python / Flask	Avancé	Architecture MVC, create_app(), sessions, middleware
SQLAlchemy / SQLite	Avancé	ORM, relations, transactions atomiques, StaticPool
HTML / CSS	Avancé	CSS Grid/Flex, variables, animations, responsive
JavaScript ES6	Intermédiaire	DOM, fetch API, calculs dynamiques temps réel
Modélisation Merise	Intermédiaire	MCD, MLD, contraintes d'intégrité, normalisation
Git / Azure DevOps	Intermédiaire	Branches, PR, CI/CD, boards Scrum
Chart.js	Intermédiaire	Bar chart, doughnut, configuration avancée
pytest	Intermédiaire	Fixtures, mocks, StaticPool, 105 tests
API REST / Webhooks	Initiation	Meta Graph API, OpenRouter, urllib, JSON
Déploiement cloud	Initiation	Render.com, Gunicorn, variables d'env prod

6.2 Compétences méthodologiques

- Agile/Scrum : maîtrise des cérémonies, rédaction de user stories, estimation, gestion de backlog.
- Analyse de l'existant : reprendre un code non documenté et le refactorer sans tout jeter.
- Conception architecturale : raisonner en couches, séparer les responsabilités.
- Gestion des priorités : arbitrer entre qualité du code, délais et attentes fonctionnelles.
- Tests first : écrire les tests comme spécification avant l'implémentation.

6.3 Compétences transversales

- Communication professionnelle : expliquer des choix d'implémentation à un non-technicien.
- Autonomie et initiative : fonctionnalités proposées sans y avoir été invitée (dashboard, calculateur).
- Résolution de problèmes : alternative email viable face au blocage WhatsApp API.
- Adaptabilité : intégration dans une équipe avec ses propres pratiques et culture.
- Curiosité technique : apprentissage autonome (OpenRouter, Meta Graph API, pytest-flask).

Réflexion personnelle : Ce stage m'a confirmé que le développement logiciel professionnel est autant une question de méthode que de technique. La rigueur dans la gestion de version, la discipline des cérémonies Scrum, et la capacité à communiquer clairement sur ses avancées sont des compétences aussi importantes que la maîtrise d'un framework.

Conclusion

Ce stage de dix semaines au sein d'Ascale a constitué une expérience fondatrice à plusieurs égards. Sur le plan technique, j'ai pu concevoir et livrer une application web complète — de la modélisation des données jusqu'aux interfaces utilisateur et aux tests unitaires — en prenant des décisions d'architecture que j'ai dû défendre, argumenter et parfois réviser face aux retours du terrain.

Sur le plan méthodologique, l'immersion dans un environnement Scrum m'a transformée dans ma façon de planifier et de communiquer. Le daily stand-up quotidien, les sprint reviews et les rétrospectives m'ont appris que le développement logiciel est avant tout une activité collective, où la transparence est aussi précieuse que la qualité du code lui-même.

Au-delà des compétences directement mobilisées, ce stage m'a offert une vision réaliste du quotidien d'un développeur en entreprise : les imprévus, la nécessité de proposer des solutions de repli, et l'art de transformer un blocage en opportunité d'apprentissage. L'intégration du fallback LLM OpenRouter pour étendre le chatbot — et celle du déploiement cloud sur Render.com — illustrent cette capacité à proposer des solutions concrètes, simples et immédiatement opérationnelles.

Je ressors de cette expérience avec la certitude d'avoir fait le bon choix de formation. Le développement web full-stack, à l'intersection de la logique back-end, de la conception de bases de données et de l'expérience utilisateur, est un domaine qui me passionne et dans lequel je souhaite continuer à progresser.

« Le meilleur code est celui qui résout un vrai problème pour de vraies personnes. »

Glossaire

Agile : Ensemble de principes de développement logiciel favorisant l'adaptation au changement, la collaboration et la livraison itérative de valeur.

API : Application Programming Interface — interface permettant à deux logiciels de communiquer (ex. OpenRouter, Meta Graph API).

Azure DevOps : Plateforme Microsoft : Boards (kanban), Repos (Git), Pipelines (CI/CD).

Back-end : Partie serveur de l'application — Python, Flask, SQLAlchemy, chatbot, emails.

Backlog : Liste priorisée de fonctionnalités à réaliser. Géré par le Product Owner.

Chart.js : Bibliothèque JavaScript open-source pour créer des graphiques dans le navigateur.

CI/CD : Intégration Continue / Déploiement Continu — automatisation des tests et du déploiement.

Daily Scrum : Réunion quotidienne de 15 min : avancement, plan du jour, blocages.

Flask : Micro-framework web Python pour créer des applications web et des API REST.

Front-end : Partie cliente de l'application — HTML, CSS, JavaScript, templates Jinja2.

Gunicorn : Serveur WSGI Python pour exposer Flask en production (multi-process).

Jinja2 : Moteur de templates Python intégré à Flask pour générer du HTML dynamique.

LLM : Large Language Model — modèle de langage IA (ex. Mistral 7B via OpenRouter).

MCD : Modèle Conceptuel de Données (Merise) — entités et associations du SI.

MVC : Modèle-Vue-Contrôleur — patron d'architecture séparant données, présentation et logique.

OpenRouter : Plateforme d'API donnant accès à plusieurs LLMs (dont des modèles gratuits).

ORM : Object-Relational Mapping — manipulation de la BDD via des objets Python (SQLAlchemy).

pytest : Framework de test Python moderne, utilisé avec pytest-flask pour les tests d'intégration.

Render.com : Plateforme cloud PaaS pour déployer des applications web (tier gratuit disponible).

Scrum : Cadre Agile en sprints de 2-4 semaines avec rôles définis et cérémonies régulières.

SQLite : Moteur BDD relationnel léger, sans serveur, stockant les données dans un seul fichier.

StaticPool : Pool SQLAlchemy partageant une connexion unique en mémoire — idéal pour les tests isolés.

Webhook : Mécanisme permettant à un service externe d'envoyer des données en temps réel via HTTP POST.

Annexes

Annexe A — Arborescence du projet

```
Ascale/
├── app.py                # Routes Flask + create_app(test_config)
├── database.py           # Modèles SQLAlchemy (8 entités)
├── models.py             # Couche service Store + seed() (20 produits)
├── chatbot.py            # Moteur chatbot (10 intentions + LLM OpenRouter)
├── email_service.py      # Emails transactionnels (Flask-Mail)
├── whatsapp_config.py    # Intégration Meta Graph API v20.0
├── requirements.txt      # Flask, SQLAlchemy, pytest, gunicorn...
├── .env.example          # Template configuration (MAIL, SECRET_KEY, OPENROUTER...)
├── .env                  # Secrets locaux (non versionné)
├── ascale.db             # Base SQLite (non versionnée)
├── tests/
│   ├── conftest.py       # Fixtures pytest (StaticPool in-memory)
│   ├── test_models.py    # 42 tests Store
│   ├── test_chatbot.py   # 27 tests chatbot
│   └── test_routes.py    # 36 tests routes Flask
├── static/
│   └── css/              # Feuilles de style custom
└── templates/
    ├── base.html         # Layout admin
    ├── base_public.html  # Layout public
    ├── accueil.html      # Page d'accueil
    ├── nos_materiaux.html # Catalogue + filtres JS
    ├── commander.html     # Commande + calculateur surface
    ├── chatbot.html       # Interface chatbot
    ├── reservation.html   # Réservation showroom
    ├── contact.html       # Formulaire contact
    ├── index.html         # Dashboard admin
    ├── commandes.html     # Gestion commandes + CSV
    ├── produits.html      # Gestion catalogue
    └── clients.html       # Gestion clients
```

Annexe B — Extrait de code : atomicité de la commande

```
def creer_commande(self, client_id, lignes_demandees, source="admin"):
    # 1. Valider TOUS les produits avant toute modification
    lignes_validees = []
    for produit_id, quantite in lignes_demandees:
        produit = Produit.query.get(produit_id)
        if quantite > produit.stock:
            raise StockInsuffisantError(f"Stock insuffisant pour {produit.nom}")
        lignes_validees.append((produit, quantite))

    # 2. Appliquer les changements seulement si tout est valide
    commande = Commande(client_id=client_id, source=source)
    db.session.add(commande)
    db.session.flush()  # obtenir l'ID sans commit

    for produit, quantite in lignes_validees:
        db.session.add(LigneCommande(
            commande_id=commande.id,
            quantite=quantite,
            prix_unitaire=produit.prix
        ))
        produit.stock -= quantite  # décrémentation atomique

    db.session.commit()
```

```
return commande
```

Annexe C — Extrait de code : fallback LLM OpenRouter

```
def _tenter_llm(store, message_original: str):
    if not OPENROUTER_API_KEY:
        return None  # désactivé si pas de clé

    # Catalogue temps réel injecté dans le system prompt
    produits = store.list_produits()
    catalogue_str = "\n".join(
        f"- {p.nom} ({p.categorie.nom}) : {p.prix:.0f} MAD/m²"
        for p in produits
    )
    payload = json.dumps({
        "model": "mistralai/mistral-7b-instruct:free",
        "messages": [
            {"role": "system", "content": _SYSTEM_PROMPT.format(catalogue=catalogue_str)},
            {"role": "user", "content": message_original},
        ],
        "max_tokens": 350,
    }).encode("utf-8")

    req = urllib.request.Request(
        "https://openrouter.ai/api/v1/chat/completions",
        data=payload,
        headers={"Authorization": f"Bearer {OPENROUTER_API_KEY}",
                 "Content-Type": "application/json"},
        method="POST",
    )
    with urllib.request.urlopen(req, timeout=10) as resp:
        return json.loads(resp.read())["choices"][0]["message"]["content"].strip()
```

Annexe D — Chaîne de priorité des intentions du chatbot

Priorité	Intention	Mots-clés déclencheurs
1	Salutation	bonjour, bonsoir, salut, hello, salam, bjr...
2	Politesse / Clôture	merci, au revoir, bonne journée, bye...
3	Action (cmd/rés.)	commander, réserver, passer commande, showroom...
4	Devis m²	regex : \d+ m² + nom produit
5	Statut commande	commande, statut, suivi, n° commande
6	Budget	budget, moins de, max, pas cher, luxe...
7	Recommandation pièce	cuisine, salle de bain, salon, terrasse, escalier...
8	FAQ Ascale	horaires, adresse, livraison, paiement, entretien...
9	Info produit	nom exact du produit (score mot ≥ 2)
10	Catégorie	marbre, granit, onyx, travertin
11	Fallback LLM	OpenRouter (si OPENROUTER_API_KEY configuré)

Annexe E — Configuration .env.example

```
# Clé secrète Flask (sessions, CSRF)
SECRET_KEY=changez-cette-valeur-en-production

# SMTP Gmail (mot de passe d'application à 16 caractères)
MAIL_SERVER=smtp.gmail.com
MAIL_PORT=587
MAIL_USE_TLS=true
MAIL_USERNAME=votre.email@gmail.com
MAIL_PASSWORD=xxxx xxxx xxxx xxxx
MAIL_DEFAULT_SENDER=Ascale Showroom <votre.email@gmail.com>
```

```
# Chatbot LLM via OpenRouter (optionnel - modèle gratuit disponible)
OPENROUTER_API_KEY=
OPENROUTER_MODEL=mistralai/mistral-7b-instruct:free

# WhatsApp Business API (Meta) - optionnel
WHATSAPP_PHONE_NUMBER_ID=
WHATSAPP_ACCESS_TOKEN=
WHATSAPP_VERIFY_TOKEN=ascale-webhook-2026
```